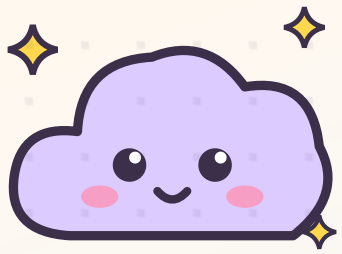
A WOWERPOINT ABOUT REALCHRISSEAN/RECALLMEM

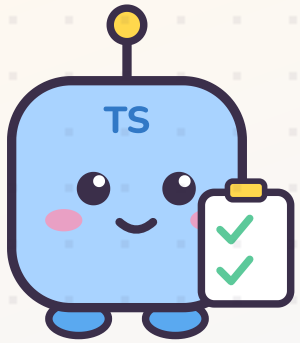
RecallMEM

A memory the model can't fake

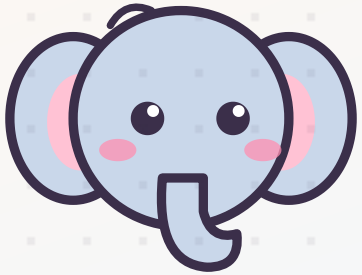
How a local-first AI remembers you, without ever letting the AI hold the pen



Sparky · the AI



Tessa · the code



Ellie · the database



You · the boss

THE PROBLEM

Chatbots have a memory problem

😶 Problem 1: they forget



😱 Problem 2: false memories (worse!)



In most chat apps with "memory," **the AI model runs its own memory.** It decides what to save, what to look up, and what to believe.

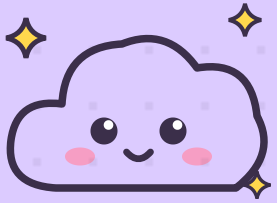
But remembering accurately is exactly what models are bad at. They hallucinate, which means confidently making things up.

Forgetting is obvious. A **confident fake memory** sounds personal, so you trust it. That's the real danger.

RecallMEM's answer: take the pen away from the AI.

MEET THE CAST

Four characters, four jobs

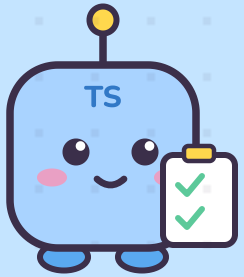


Sparky

the AI model

Claude, GPT, Grok, or a local Gemma 4. Brilliant and chatty, but sometimes makes things up.

Job: PROPOSES

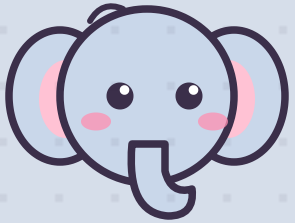


Tessa

the TypeScript code

The plain code in RecallMEM's `lib/` folder. She checks every claim against hard rules.

Job: DECIDES



Ellie

Postgres + pgvector

A local database. pgvector lets it find text by *meaning*, not just exact words.

Job: REMEMBERS



You

the human

You can see, edit, and delete every memory, and you sign off on the sensitive ones.

Job: FINAL SAY

THE GOLDEN RULE

The LLM proposes. TypeScript decides.

(LLM = large language model, the AI that writes the replies)

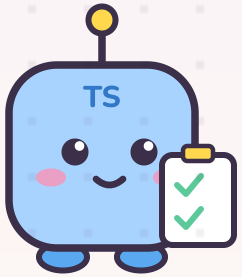


Sparky CAN ✓

- ✓ Suggest new facts about you
- ✓ Point out facts that are no longer true
- ✓ Write the actual replies

Sparky CAN'T ✕

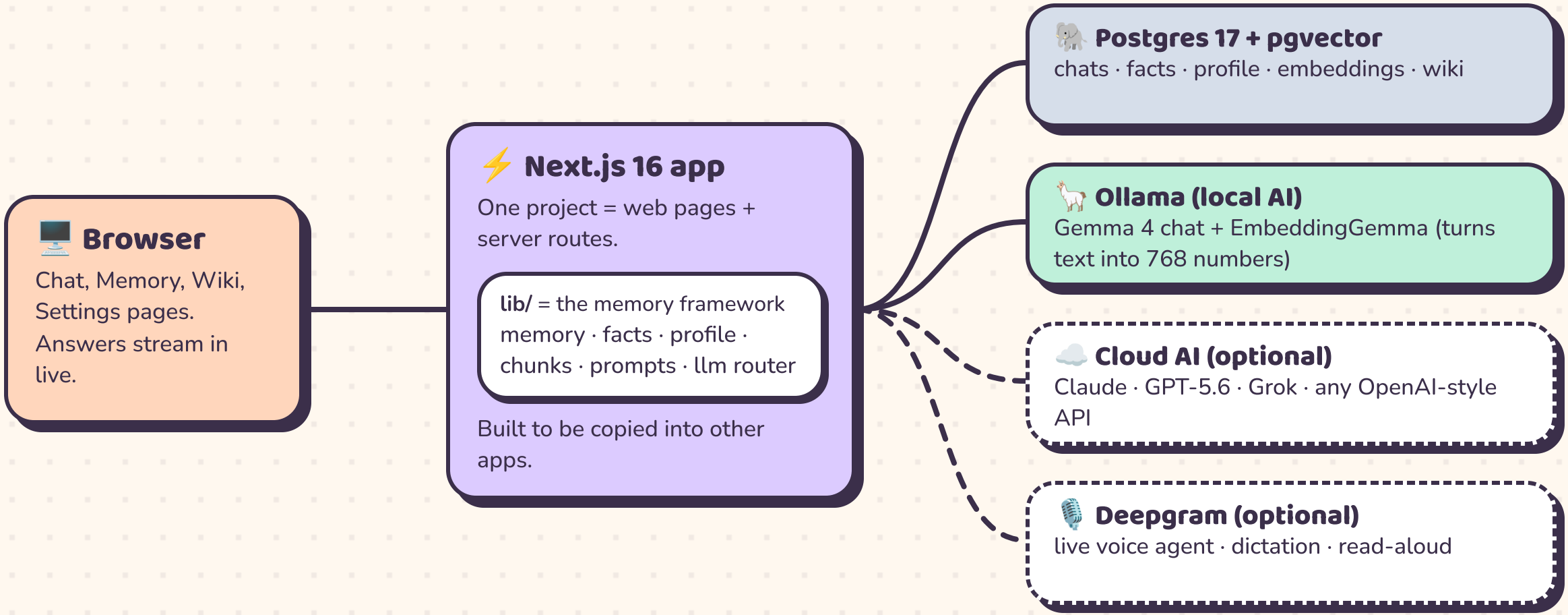
- ✕ Write anything to the database
- ✕ Choose which memories get pulled up
- ✕ See any memory it wasn't handed



WHAT'S IN THE BOX

Everything runs on your machine

One command: `npx recallmem` → opens at `localhost:1337`



THE MEMORY ITSELF

Three layers of memory (plus your rules)

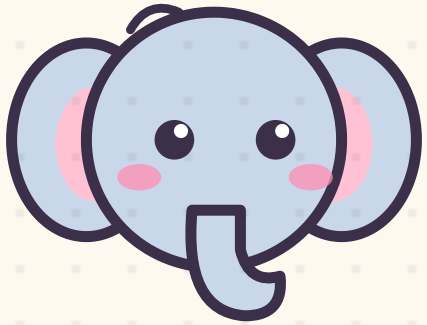
1. **Profile: "who am I talking to?"**
   Goes into every chat. **Built by code, not AI:** active facts grouped under IDENTITY, FAMILY, WORK, HEALTH..., with a cap per heading so it can't grow forever.

2. **Facts: small, checkable statements**
   One database row each: *"User works at Acme"* + category + date it became true + date it stopped + status + **the exact quote that proves it**.

3. **Recalled chunks: old conversations**
   Past chats cut into ~1,000-character pieces and searchable by meaning. Each is labeled `[from conversation on 2026-03-12]`.

+ **RULES.md: your standing instructions**
  "Don't add disclaimers." "Pick one option." These override the defaults in every chat.

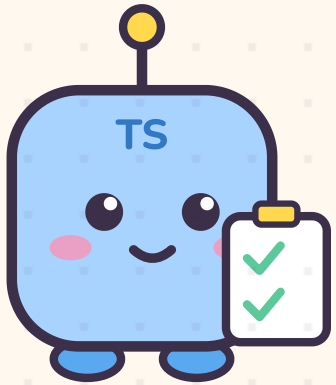
I keep all three, with dates on everything!



THE READ PATH

Memory is gathered *before* Sparky wakes up

1. **Safety limits**: at most 2,000 messages, 1M characters, 20 images. Stops runaway bills.
2. **4 lookups at once:** profile · facts · time since last chat · your rules
3. **Smart facts, up to 150 slots:** 30 "who you are" facts → 50 related to your message → newest facts fill the rest
4. **Old conversation search:** top 8 passages, with a **3-second deadline**. Too slow? Chat goes on without them.
5. **Date-stamp everything** so the model can tell "then" from "now"
6. **A code template** fills in the system prompt (the hidden instructions sent before your message)
7. **Only now** is the AI model called



Sparky never touches the database. It can't "remember" what isn't in the prompt.

Bonus: you can trace exactly which facts went into any answer.

HOW ELLIE FINDS THINGS

Hybrid search: meaning + exact words

You type: `what went wrong with grok-4.20-0309-reasoning?`

🔤 Lane 1: Exact words

- Postgres full-text search + a literal "contains" match
- Special characters like `%` and `_` are escaped so the search stays literal
- Hit inside a fact's proof quote? Tagged **receipt** . Otherwise **keyword**

+

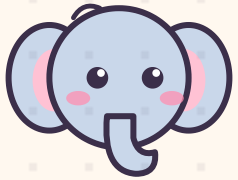
🧭 Lane 2: Meaning

- Your message becomes an embedding (a list of numbers that captures meaning)
- pgvector finds the stored ones closest to it
- Tagged **semantic** , kept only if genuinely close

=

🧺 Merged

- Exact hits first, meaning hits after
- Duplicates dropped
- Embedding service down? Exact search still works



Meaning finds ideas. Exact words find model IDs, branch names, and error messages.

DURING THE CHAT

Long chats stay cheap, private, and aware

🪟 Sliding window

Over 10 messages? Only the **last 5 back-and-forths** go in full. Older parts of *this* chat are found by meaning (top 3). Cost stays flat however long you talk.

⏰ "Welcome back" marker

Back after 2+ hours? A note is slipped in first: `[Conversation resumed 6 days later, on 2026-05-23.]` so the AI knows time has passed.

🔒 Private mode

One click and **no profile, facts, or old chats** are sent to the AI. Only your rules and your message. Nothing from that chat gets saved as memory.

🌐 Web search

Claude uses its built-in search. Other models use **Brave Search**: RecallMEM searches, then pastes results above the prompt. Brave sees your message, **never your memory**.

💾 **Never lose an answer:** replies stream in live, the partial answer is saved every 3 seconds, and the browser keeps a backup copy in case something breaks mid-reply.

THE WRITE PATH · PART 1

Sparky proposes… with a receipt

1. After **every reply**, a background job starts. You never wait for it.
2. It picks the **cheapest model you have**: Claude Haiku → GPT-5.6 Luna → Grok Mini → free local Gemma
3. It reads only the **latest exchange** (~2K tokens), your active facts with their IDs, and **today's date**
4. Sparky answers in strict JSON (a structured data format). **Every fact needs a quote:**

```
{"facts": [{
  "text":  "User lives in Example City",
  "quote": "I live in Example City",
  "supersedes": ["id-of-an-old-fact"]
}]}
```

No quote, no memory.



🗓️ Times must become real dates

You say: *"My job starts in 1 month."*

× ~~User's job starts in 1 month~~ (gets more wrong every day)

✓ User said on **2030-01-15** that their job starts around **2030-02-15**

THE WRITE PATH · PART 2

Tessa's five gates

Every proposed fact must pass all five, or it's quietly thrown away

1
Shape
Must have both a fact *and* a quote. Plain text without proof ×

2
Junk filter
"User asked about...", "AI suggested...", "not mentioned", under 10 characters ×

3
Time
"soon", "next week", "currently" with no real date ×

4
Receipt
The quote must **really appear** in the chat (ignoring case and spacing, 8+ characters)

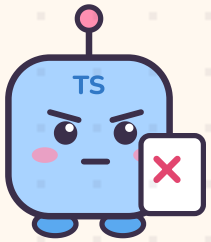
5
Real IDs
"This replaces fact X" only counts if X is one of your real active facts



"User collects antique maps"
quote: "I collect antique maps."

→

That quote isn't in the chat. Rejected. ×



"The model can still hallucinate. It just can't make the database believe it."

THE WRITE PATH · PART 3

Trust levels: what gets used right away

🧹 Dedupe
Same as a fact you already have? Dropped.

→

🏷️ Sort by keyword, not AI
"wife", "dog" → family · "hired" → work. Smart about word forms, so **"son" ≠ "Sonnet"** and **"work" ≠ "framework"**.

→

⚖️ Trust rule
Decides active or pending (below)

✅ Active right away
Low-risk facts: projects, interests, preferences, social, work. They join the profile right away, so your next message already knows.

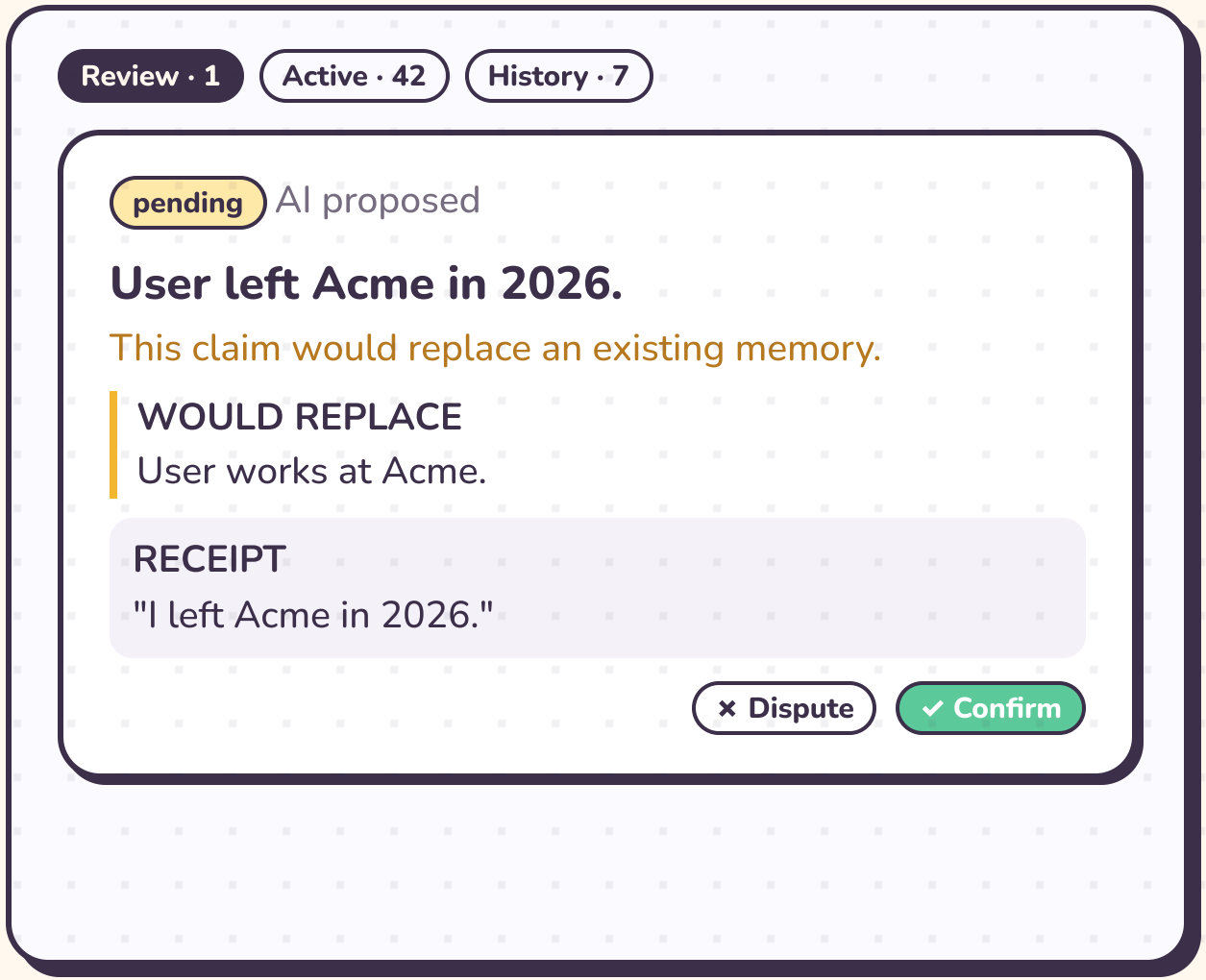
⏳ Pending your review
Sensitive topics: identity, family, money, health.
Anything that would replace an existing memory. The old fact keeps working until *you* confirm the swap.



Clicking "New chat" = the bigger save. The app shows "Saving memory…" while it names the chat, checks the *whole* conversation for facts, rebuilds the profile, and makes the chat searchable. (It used to run in the background, but the next chat sometimes started before memories were saved.)

YOU'RE THE BOSS

The Memory page: every claim, with its receipt



Three tabs
Review: confirm or dispute · **Active**: edit or retire · **History**: restore or delete forever

History is kept, not overwritten
Confirming a swap retires the old fact, stamps its end date, and links it to the new one. Every change happens in one safe database step.

🧨 **Danger zone**
Wipe memory or **Delete everything** (type DELETE). Then Postgres physically rewrites its files so the data can't be recovered.

MULTIPLE BRAINS

Brains: separate memory spaces

🧠 Default · 💼 Work · 🏠 Personal · 🎯 Demo · 📚 Project wiki

🍪 Pick a brain

Saved in a browser cookie: `recallmem-brain=work`

→

🪪 Your ID changes

`local-user` becomes `local-user::work`

→

🔐 Everything is scoped

Chats, facts, profile, and searches only see that brain

🤝 Shared across all brains

API keys, settings, and rules. Add your Claude key once and every brain can use it.

🛠️ For developers

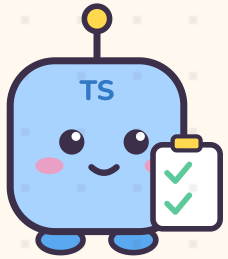
The same idea makes `lib/` reusable: call `configureDb()` and `setUserIdResolver()` with your own login system and every function becomes per-user.

WIKI & STUDY MODE

Cite it, or say "I don't know"

A brain can become a documentation assistant that answers *only* from sources you add

1. **Add sources**
   Paste text, upload a file, or import a public GitHub repo (up to 120 files, with the exact commit saved)
2. **Chunk with line numbers**
   ~1,400-character pieces split at headings, each remembering lines `L10–L24`
3. **Search + answer**
   Top 8 chunks go to the model, which must reply with citations: chunk, lines, exact quote
4. **Tessa verifies**
   Chunk exists? Lines inside it? Quote really there? Bad citations are dropped
5. **Clickable proof**
   GitHub citations link straight to the file, commit, and line numbers



Nothing matches? → "I don't have that in this brain's sources." (no AI call at all)

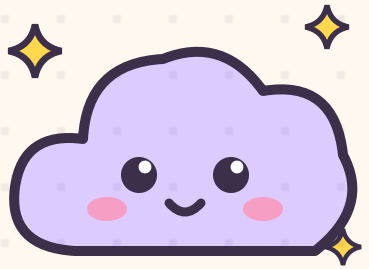
No valid citation survives? It won't show an unverified answer. It shows the raw excerpts instead, clearly labeled. **Study Mode** uses the same checks but teaches by asking a question first.

LIVE VOICE AGENT

Talk out loud. Same memory, same rules.

- 🔑 **Safe keys:** the browser gets a 5-minute temporary pass, never your real Deepgram key
- ⚡ **Fast models only:** local Gemma is too slow for live talk, so only quick cloud models are allowed
- 👂 **Better hearing:** names and project terms from your memory (up to 80) help the speech recognizer spell them right
- 🔎 **Memory as a tool:** the agent can call `search_memory`. Exact matches come back first; meaning matches only if ready within 0.9 s
- 🛟 **Backups:** if the model or voice fails mid-call, a fallback model or voice takes over
- 💾 **One memory:** each spoken turn is saved as a normal chat message and goes through the same fact checks

Let me check your memory for that.



Said automatically if a lookup takes over 0.45 s, so there's no awkward silence.

THE REST OF THE PRODUCT

Small touches that make it real

📦 One-command install

A tiny 22 KB installer: on a Mac it sets up Postgres and Ollama with one yes/no question. `doctor` finds problems, `upgrade` updates.

🎛️ Model picker

Local Gemma 4 in 4 sizes with a download bar. GPT-5.6 and Claude in different thinking depths, with prices shown.

📄 PDFs as pictures

Each page (up to 20) is turned into an image, so the AI can read charts and scans, not just the text.

🎤 Dictate & listen

Speak your message, or have replies read aloud by Deepgram, xAI, OpenAI, or your browser's built-in voice.

💸 Cost tracker

Every AI, speech, and transcription call is logged with an estimated price, by day, week, or month.

🛡️ Security basics

API keys encrypted on disk. Blocks provider URLs that point at your internal network. Self-update only works from your own machine.

🧪 **Tests where it counts:** automated tests cover the receipt check, date rules, keyword sorting ("Sonnet" bug included), the trust rule, search fallbacks, wiki citations, encryption, and the voice setup. A secret scanner blocks API keys and database dumps from being committed.

HONEST NOTES FROM READING EVERY FILE

Rough edges & what's next

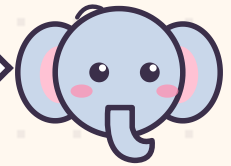
📝 Docs that fell behind the code

- Architecture doc says extraction "always uses the local model". The code now prefers a cheap cloud model if you have a key
- README mentions a "6-step validator". The code has since added receipts, date rules, and a review inbox
- The profile is called "synthesized", but it's built by code with no AI (arguably a strength)
- CONTRIBUTING says there are no tests. There are now
- The Gmail/Notion/Calendar "Connections" panel is a preview behind "Coming Soon"

🗺️ On the roadmap

- Stronger links from a retired fact to the fact that replaced it
- A test suite that measures memory quality, to catch regressions
- Entity memory: linking people, companies, projects, and places
- Showing *why* a memory was used (exact match? similar meaning?)
- Export/import everything from Settings
- An always-on hosted version, with login added first

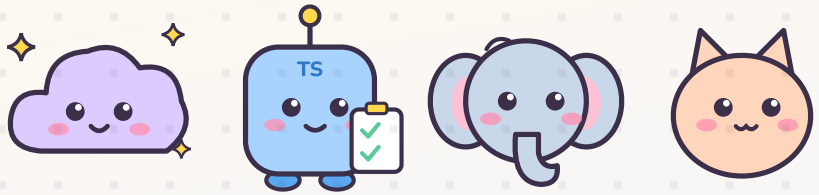
Normal for a fast-moving, one-person project!



THE BIG IDEA

One rule, repeated at every layer

Where	🌥️ What Sparky (the AI) does	🤖 What Tessa (the code) does
Reading memory	Nothing. It gets a finished prompt	Database lookups + search + a fill-in template
Proposing a fact	Suggests text, a quote, and replacements	Checks shape, junk, dates, the quote, and IDs
Saving a fact	Nothing	Dedupes, sorts by keyword, applies the trust rule
Replacing a fact	Points out what seems out of date	Holds it for **your** approval
Wiki answers	Writes an answer with citations	Checks every chunk, line range, and quote



Missing a memory is annoying.
Storing a false one is worse.